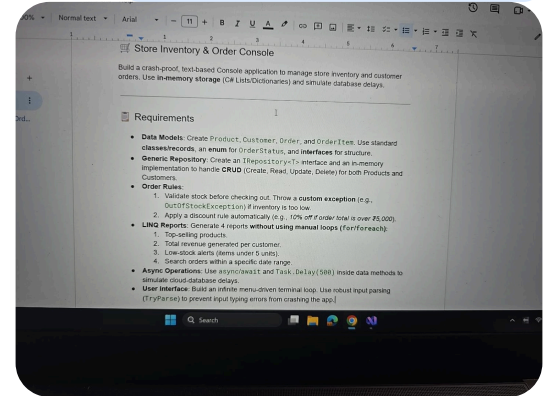


C# Inventory Project

Exported August 20, 2026 • 22 messages

YOU

Explain me the complete process



CHATGPT

Absolutely. This assignment is essentially asking you to build a **C# console-based mini e-commerce/store management system** using OOP, generics, LINQ, async/await, exception handling, and collections.

The best way to understand it is to build it in layers.

1. What are we actually building?

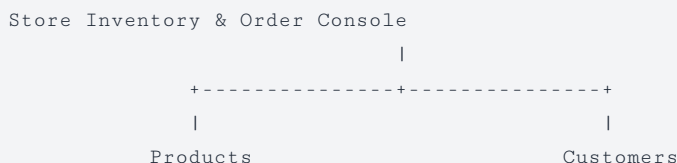
The application is called **Store Inventory & Order Console**.

Think of it like a very small version of Amazon's backend, but everything is stored **in memory** instead of a real database.

The application should allow you to:

- Add/manage products
- Add/manage customers
- Create customer orders
- Check whether enough stock exists
- Automatically apply discounts
- Reduce inventory after checkout
- Generate reports using LINQ
- Simulate database/network delay using `Task.Delay(500)`
- Never crash because of invalid user input

The overall flow is:



```

      |                                     |
      +-----+-----+-----+
      |
      | Orders
      |
      | Validate Stock
      |
      | Calculate Total
      |
      | Apply Discount
      |
      | Update Inventory
      |
      | Save Order
      |
      | LINQ Reports

```

2. Technologies/concepts required

Your assignment is testing several C# concepts at once:

Requirement	C# Concept
Product, Customer, Order	Classes / Records
OrderStatus	Enum
Structure/interfaces	Interfaces
Generic Repository	Generics
In-memory storage	<code>List<T></code> , <code>Dictionary<TKey,TValue></code>
CRUD	Create, Read, Update, Delete
Stock validation	Custom Exception
Discount	Business logic
Reports	LINQ
Database simulation	<code>async/await</code> , <code>Task.Delay()</code>
Crash-proof input	<code>TryParse()</code>
Menu	<code>while</code> loop / switch
Data abstraction	Repository + Service

So don't think of this as one giant program.

Think of it as **multiple layers**.

3. Recommended project structure

Create a C# Console project.

For example:

```

StoreInventoryConsole/
|
├── Models/
|   ├── Product.cs
|   ├── Customer.cs
|   ├── Order.cs
|   ├── OrderItem.cs
|   └── OrderStatus.cs
|
├── Interfaces/
|   └── IRepository.cs
|
├── Repositories/
|   └── InMemoryRepository.cs
|
├── Exceptions/
|   └── OutOfStockException.cs
|
├── Services/
|   └── StoreService.cs
|
├── Reports/
|   └── ReportService.cs
|
└── Program.cs

```

You don't strictly have to create this many folders, but this structure makes the assignment much easier to understand.

4. Step 1 — Create the Product model

A product represents something being sold.

For example:

```

Product
-----
Id
Name
Price
StockQuantity
Category

```

Example:

```

Product
Id: 101
Name: Laptop
Price: ₹60,000
Stock: 10

```

In C#:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
    public string Category { get; set; } = "";
}
```

Why `decimal` for Price?

Don't normally use `double` for money.

Use:

```
decimal
```

because it provides better precision for financial calculations.

5. Step 2 – Create Customer

A customer represents someone who places an order.

```
Customer
-----
Id
Name
Email
Phone
```

Example:

```
Customer
Id: 1
Name: Shreyash
Email: shreyash@gmail.com
```

C#:

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Email { get; set; } = "";
    public string Phone { get; set; } = "";
}
```

6. Step 3 – Create OrderItem

This is extremely important.

An `Order` can contain multiple products.

For example:

```
Order #1001

Laptop      × 1
Mouse       × 2
Keyboard    × 1
```

Each individual line is an `OrderItem`.

```
OrderItem
-----
ProductId
Quantity
UnitPrice
```

For example:

```
ProductId = 101
Quantity = 2
UnitPrice = ₹500
```

C#:

```
public class OrderItem
{
    public int ProductId { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}
```

7. Step 4 – Create OrderStatus enum

The assignment specifically says:

use an enum for OrderStatus

An order can have states such as:

```
Pending
Confirmed
Cancelled
Completed
```

So:

```
public enum OrderStatus
{
    Pending,
    Confirmed,
    Cancelled,
    Completed
}
```

Now instead of writing:

```
order.Status = "Completed";
```

you write:

```
order.Status = OrderStatus.Completed;
```

This is safer because invalid values such as "xyz" aren't accepted as a status.

8. Step 5 — Create Order

An order connects the customer and products.

It might contain:

```
Order
-----
Id
CustomerId
Items
OrderDate
TotalAmount
Discount
Status
```

For example:

```
Order #5001

Customer: Shreyash
Date: 18-Aug-2026

Items:
    Laptop      × 1    ₹60,000
    Mouse       × 2    ₹1,000

Subtotal: ₹61,000
Discount: ₹6,100
Total: ₹54,900

Status: Completed
```

C#:

```
public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }

    public List<OrderItem> Items { get; set; } = new();

    public DateTime OrderDate { get; set; }

    public decimal TotalAmount { get; set; }
```

```

    public decimal Discount { get; set; }

    public OrderStatus Status { get; set; }
}

```

9. Step 6 – Create IRepository<T>

This is where **generics** come in.

The requirement says:

Create an `IRepository<T>` interface and an in-memory implementation to handle CRUD for both Products and Customers.

Instead of creating:

```

ProductRepository
CustomerRepository

```

separately, create one generic repository.

Conceptually:

```

IRepository<T>
|
+---- Product
|
+---- Customer

```

The interface defines CRUD operations.

```

public interface IRepository<T>
{
    Task AddAsync(T item);
    Task<T?> GetByIdAsync(int id);
    Task<List<T>> GetAllAsync();
    Task UpdateAsync(T item);
    Task DeleteAsync(int id);
}

```

Notice the `<T>`.

`T` means:

"I don't know the type yet. Give me the type when you use this interface."

10. How generic repository works

You can create:

```

IRepository<Product>

```

which means:

```
T = Product
```

Or:

```
IRepository<Customer>
```

which means:

```
T = Customer
```

Therefore the same repository implementation can handle both.

11. Step 7 – In-memory repository

Instead of a real database, use:

```
List<T>
```

For example:

```
private readonly List<T> items = new();
```

So your "database" is basically:

```
Memory
|
├── Products List
|
├── Customers List
|
└── Orders List
```

Nothing is stored permanently.

If the application closes, the data disappears.

That's exactly what the assignment means by:

Use in-memory storage.

12. Why `Task.Delay(500)`?

The assignment says:

simulate database delays.

A real application might do:

```
Application
  ↓
Database
  ↓
Network
```

```

↓
Database processing
↓
Response

```

That takes time.

But our application has no database.

So we simulate that delay:

```
await Task.Delay(500);
```

Meaning:

Pretend the database took 500 milliseconds to respond.

13. Why async/await?

Suppose we have:

```

public async Task<List<T>> GetAllAsync()
{
    await Task.Delay(500);

    return items.ToList();
}

```

The method is asynchronous.

Instead of:

```
List<T>
```

we return:

```
Task<List<T>>
```

And call it with:

```
var products = await repository.GetAllAsync();
```

So the flow is:

```

GetAllAsync()
↓
Task.Delay(500)
↓
Pretend database responded
↓
Return products

```

14. CRUD operations

CRUD means:

```
C → Create
R → Read
U → Update
D → Delete
```

Your repository should support all four.

Create

```
Add Product
    ↓
Repository
    ↓
List<Product>.Add()
```

Read

```
Get Product
    ↓
Repository
    ↓
Find by ID
```

Update

```
Existing Product
    ↓
Modify properties
    ↓
Repository
```

Delete

```
Product ID
    ↓
Find product
    ↓
Remove from List
```

15. Step 8 – Custom exception

The assignment specifically says:

Throw a custom exception if inventory is too low.

Create:

```
public class OutOfStockException : Exception
{
    public OutOfStockException(string message)
        : base(message)
```

```
{
  }
}
```

Now imagine:

```
Product stock = 3

Customer wants = 5
```

You cannot fulfill the order.

Instead of silently failing:

```
throw new OutOfStockException(
    "Insufficient stock for the requested product."
);
```

16. Stock validation is VERY important

The assignment says:

Validate stock before checking out.

This means you should **not** reduce stock first.

Bad approach:

```
Product stock = 3

Reduce stock
  ↓
stock = -2
  ↓
Oops! Customer wanted 5
```

Correct approach:

```
Customer places order
  ↓
Check ALL items
  ↓
Enough stock?
  ↙       ↘
NO         YES
  ↓         ↓
Exception Checkout
              ↓
            Reduce inventory
```

17. Why check ALL products first?

Suppose an order contains:

```
Laptop × 1
Mouse × 2
Keyboard × 5
```

Stock:

```
Laptop = 10
Mouse = 10
Keyboard = 2
```

Keyboard is insufficient.

You should **not** reduce:

```
Laptop → 9
Mouse → 8
```

and then discover keyboard doesn't have enough.

Instead:

```
Validate Laptop ✓
Validate Mouse ✓
Validate Keyboard ✗

Cancel entire checkout
```

This gives you safer order processing.

18. Step 9 – Calculate order total

For each item:

```
Price × Quantity
```

Suppose:

```
Laptop = ₹60,000 × 1
Mouse = ₹500 × 2
```

Then:

```
Subtotal
= 60,000 + 1,000
= ₹61,000
```

Using LINQ:

```
decimal subtotal = order.Items
    .Sum(item => item.UnitPrice * item.Quantity);
```

Notice this uses LINQ rather than a manual loop.

19. Step 10 – Apply discount

The requirement says:

10% off if order total is over ₹5,000.

So:

Subtotal = ₹6,000

Discount = 10%
= ₹600

Final = ₹5,400

Conceptually:

```
if (subtotal > 5000)
{
    discount = subtotal * 0.10m;
}
else
{
    discount = 0;
}

total = subtotal - discount;
```

Important:

"over ₹5,000" means `> 5000`, not `>= 5000`.

So exactly ₹5,000 gets **no discount** according to the wording.

20. Step 11 – Checkout process

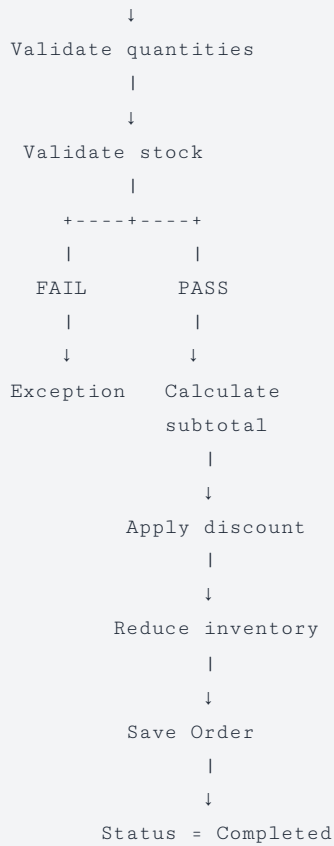
This is the heart of the application.

When the user selects:

Checkout

the process should be:

```
CHECKOUT
      |
      ↓
Get customer
      |
      ↓
Get order items
      |
      ↓
Validate products
      |
```



This ordering is important.

21. Step 12 – LINQ Reports

The assignment requires **four reports**.

And specifically says:

without using manual loops (`for/foreach`)

So this is where you demonstrate LINQ.

Report 1 – Top-selling products

Question:

Which products have sold the most?

Suppose orders contain:

```

Order 1:
Laptop × 2
Mouse × 3

Order 2:
Laptop × 1
Mouse × 1
  
```

```
Order 3:
Keyboard * 5
```

Sales:

```
Laptop → 3
Mouse → 4
Keyboard → 5
```

Therefore:

```
Keyboard
Mouse
Laptop
```

You can use:

```
SelectMany
GroupBy
Sum
OrderByDescending
```

The important concept is:

```
Orders
↓
Items
↓
Group by ProductId
↓
Sum quantities
↓
Sort descending
```

22. Report 2 – Total revenue per customer

Question:

How much money has each customer generated?

Example:

```
Shreyash → ₹25,000
Pradnya → ₹17,500
Anushka → ₹31,000
```

Conceptually:

```
Orders
↓
GroupBy CustomerId
↓
Sum TotalAmount
```

↓

Order/Display

LINQ uses:

```

    GroupBy ( )
    Sum ( )

```

23. Report 3 – Low-stock alerts

Requirement:

items under 5 units

Suppose:

Laptop	10
Mouse	3
Keyboard	2
Monitor	7

The report should show:

```

    Mouse
    Keyboard

```

LINQ:

```

    Products
    ↓
    Where (stock < 5)
    ↓
    Display

```

This demonstrates:

```

    Where ( )

```

24. Report 4 – Search orders by date range

Suppose the user enters:

```

    From: 01-Aug-2026
    To:   18-Aug-2026

```

You filter:

```

    Orders
    ↓
    OrderDate >= From
    AND
    OrderDate <= To

```

↓

Matching orders

LINQ:

Where()

For example conceptually:

```
orders.Where(o =>
    o.OrderDate >= from &&
    o.OrderDate <= to
);
```

25. Important LINQ methods you should understand

Your faculty may ask you about these.

Where()

Filters data.

```
products.Where(p => p.StockQuantity < 5)
```

Means:

Give me products whose stock is below 5.

Select()

Transforms data.

```
products.Select(p => p.Name)
```

Means:

Give me only product names.

SelectMany()

Flattens nested collections.

This is especially useful for:

```
Orders
↓
OrderItems
```

because every order contains multiple items.

GroupBy()

Groups related data.

```
Orders
↓
Group by CustomerId
```

Sum()

Adds values.

```
items.Sum(x => x.Quantity)
```

OrderByDescending()

Sorts largest → smallest.

```
100
80
50
20
```

FirstOrDefault()

Gets the first matching element or returns null/default.

26. Step 13 – User Interface

The assignment requires:

infinite menu-driven terminal loop

So your program shouldn't just run once.

It should continuously show:

```
=====
      STORE INVENTORY SYSTEM
=====

1. Add Product
2. View Products
3. Update Product
4. Delete Product

5. Add Customer
6. View Customers
7. Update Customer
8. Delete Customer

9. Create Order
10. Checkout

11. Top Selling Products
12. Revenue Per Customer
13. Low Stock Alerts
```

```
14. Search Orders By Date
```

```
0. Exit
```

```
Enter choice:
```

Then:

```
while (true)
{
    DisplayMenu();

    string? input = Console.ReadLine();

    ...
}
```

The loop continues until:

```
0 → Exit
```

27. Step 14 – Why `TryParse()` ?

This is another important requirement.

Suppose you ask:

```
Enter product price:
```

and the user types:

```
hello
```

If you do:

```
decimal price = decimal.Parse(Console.ReadLine());
```

the application can crash.

Instead:

```
if (decimal.TryParse(input, out decimal price))
{
    // valid
}
else
{
    Console.WriteLine("Invalid price.");
}
```

Same for integers:

```
int.TryParse()
```

So:

```

User input
  ↓
TryParse()
  ↓
Valid?
  ↙   ↘
YES    NO
  ↓     ↓
Continue Show error

```

This is what they mean by:

crash-proof

28. Example of robust input

Instead of:

```
int choice = int.Parse(Console.ReadLine());
```

use:

```

if (!int.TryParse(Console.ReadLine(), out int choice))
{
    Console.WriteLine("Please enter a valid number.");
    continue;
}

```

Now the application doesn't die when someone types:

abc

29. Complete architecture

Your final architecture should look something like this:

```

PROGRAM.CS
      |
      ↓
Menu / UI Layer
      |
      ↓
StoreService
      |
+-----+-----+
|               |
|               |
↓               ↓
Repositories    ReportService
|               |
↓               ↓
In-Memory Lists LINQ

```

```

      |
      ↓
Product / Customer
      / Order Data

```

And exceptions:

```

StoreService
  |
  ├── OutOfStockException
  |
  └── Input validation

```

30. What each layer does

Models

Represent data.

```

Product
Customer
Order
OrderItem
OrderStatus

```

They shouldn't contain lots of application logic.

Interfaces

Define contracts.

```

IRepository<T>

```

It says:

Any repository must provide CRUD operations.

Repository

Handles data storage.

```

Add
Get
GetAll
Update
Delete

```

Currently:

```

List<T>

```

Later, it could theoretically be changed to:

```
SQL Server
MongoDB
PostgreSQL
```

without changing the entire application.

That's the advantage of the repository pattern.

Service

Contains business rules.

For example:

```
Checkout
Stock validation
Discount
Order creation
Inventory update
```

This is where the important application logic belongs.

ReportService

Contains LINQ queries.

```
Top selling
Revenue
Low stock
Date search
```

Program

Handles:

```
Menu
User input
Calling services
Displaying results
```

31. Complete example scenario

Let's see how the application behaves from beginning to end.

Start application

```
STORE INVENTORY & ORDER CONSOLE

1. Add Product
2. Add Customer
3. Create Order
```

```
4. Reports
0. Exit
```

Add product

User enters:

```
ID: 101
Name: Laptop
Price: 60000
Stock: 10
```

System:

```
Product added successfully.
```

Memory now contains:

```
Products:

101 | Laptop | ₹60,000 | Stock: 10
```

Add customer

```
ID: 1
Name: Shreyash
Email: shreyash@gmail.com
```

Memory:

```
Customers:

1 | Shreyash | shreyash@gmail.com
```

Create order

Customer:

```
1
```

Product:

```
101
```

Quantity:

```
1
```

System checks:

```
Required = 1
Available = 10
```

So:

```
Stock ✓
```

Calculate

```
Laptop
₹60,000 × 1

Subtotal = ₹60,000
```

Discount:

```
₹60,000 > ₹5,000

10% discount = ₹6,000
```

Final:

```
₹60,000 - ₹6,000
= ₹54,000
```

Update inventory

Before:

```
Laptop = 10
```

After:

```
Laptop = 9
```

Save order

```
Order ID: 1001
Customer: Shreyash
Total: ₹54,000
Status: Completed
```

32. What happens if stock is insufficient?

Suppose:

```
Laptop stock = 2
```

Customer wants:

5

Process:

```
Requested = 5
Available = 2
```

Condition:

$5 > 2$

Therefore:

```
throw new OutOfStockException(...)
```

The UI catches it:

```
ERROR:
Insufficient stock for Laptop.

Available: 2
Requested: 5
```

The application **continues running**.

This is important.

A custom exception shouldn't cause the whole application to crash because you've handled it appropriately.

33. Exception handling structure

At the UI level you can have:

```
try
{
    await service.CheckoutAsync(...);
}
catch (OutOfStockException ex)
{
    Console.WriteLine(ex.Message);
}
catch (Exception ex)
{
    Console.WriteLine("Something went wrong.");
}
```

So:

```
Exception
↓
Catch
↓
Display friendly message
```



Return to menu

34. Why use interfaces?

The assignment also mentions:

interfaces for structure

For example:

```
IRepository<T>
```

is an abstraction.

The service doesn't need to know:

"Am I using List<T>, SQL Server, MongoDB, etc.?"

It only knows:

```
IRepository<Product>
```

This is an important OOP principle:

Program against abstractions, not concrete implementations.

35. Dependency relationship

Conceptually:

```
Program
  ↓
StoreService
  ↓
IRepository<Product>
  ↓
InMemoryRepository<Product>
```

and:

```
Program
  ↓
StoreService
  ↓
IRepository<Customer>
  ↓
InMemoryRepository<Customer>
```

This is much cleaner than putting everything inside `Program.cs`.

36. How the async flow looks

For example:

```

Program
  |
  | await AddProductAsync()
  ↓
StoreService
  |
  | await repository.AddAsync()
  ↓
Repository
  |
  | Task.Delay(500)
  ↓
Memory List

```

Then control returns back:

```

Memory List
  ↓
Repository
  ↓
Service
  ↓
Program
  ↓
"Product added successfully."

```

37. One important design decision: Orders repository

The requirement explicitly says CRUD for:

```

Products
Customers

```

It doesn't necessarily require CRUD for orders.

However, since reports need orders, you still need to store them somewhere.

So you can maintain:

```

IRepository<Product> productRepository;
IRepository<Customer> customerRepository;
IRepository<Order> orderRepository;

```

This is actually cleaner.

38. Suggested initialization

When the program starts, you can create:

```

Product Repository
Customer Repository
Order Repository

```



You can also add some sample data so that reports aren't empty when demonstrating the project.

For example:

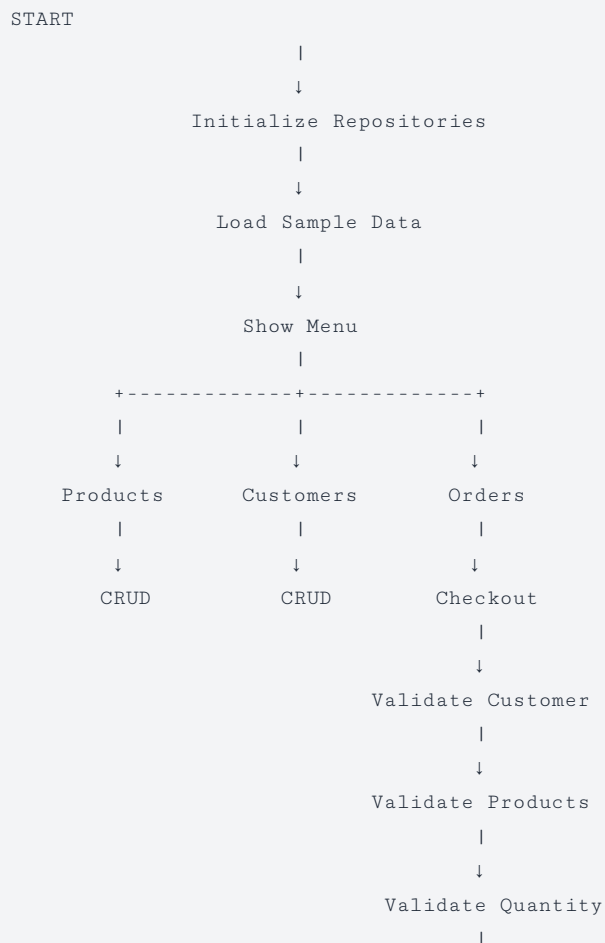
```
Products:
Laptop
Mouse
Keyboard
Monitor

Customers:
Shreyash
Pradnya
Anushka
```

Then create several orders.

39. Complete application flow

Putting **everything** together:





40. Recommended implementation order

Don't try to write everything at once.

Build it in this exact order:

Phase 1 — Project

Create Console App

Phase 2 – Models

Create:

```
Product
Customer
OrderItem
Order
OrderStatus
```

Phase 3 – Repository

Create:

```
IRepository<T>
InMemoryRepository<T>
```

Implement:

```
Create
Read
Read All
Update
Delete
```

Add:

```
async/await
Task.Delay(500)
```

Phase 4 – Exception

Create:

```
OutOfStockException
```

Phase 5 – Service

Implement:

```
Add Product
Update Product
Delete Product
Add Customer
Update Customer
Delete Customer
Checkout
```

Phase 6 – Checkout

Implement in this order:

```
Find customer
    ↓
Find products
```

```

↓
Validate stock
↓
Calculate subtotal
↓
Apply 10% discount
↓
Reduce stock
↓
Create order
↓
Save order

```

Phase 7 – Reports

Implement the four LINQ reports:

1. Top-selling products
2. Revenue per customer
3. Low-stock products
4. Orders by date range

Phase 8 – UI

Create menu:

```
while(true)
```

Add:

```
TryParse()
try/catch
```

Phase 9 – Testing

Test:

```

Valid product
Invalid product input
Duplicate ID
Update product
Delete product
Valid checkout
Insufficient stock
Discount below ₹5,000
Discount above ₹5,000
Multiple order items
Low-stock report
Revenue report
Top-selling report
Date-range report
Invalid menu input

```

41. What your faculty is actually testing

This assignment isn't primarily about making a fancy store application.

They're testing whether you understand these C# concepts:

```
C# Concepts

      |
+-----+-----+
|         |         |
|  OOP    |  Generics  |  LINQ
|         |         |         |
|  Classes | IRepository | Reports
|  Records | Repository |
| Interfaces|
|         |
+-----+-----+
|         |
| Exception Handling
|         |
| async / await
|         |
| Task.Delay()
|         |
| TryParse / Console
```

If you understand **why each component exists**, you'll be able to explain the project in a viva instead of just showing code.

The most important mental model

Remember these five responsibilities:

```
MODEL
"What data do I have?"

REPOSITORY
"Where is that data stored?"

SERVICE
"What rules should the application follow?"

REPORT
"What information can I derive using LINQ?"

PROGRAM/UI
"How does the user interact with it?"
```

So if your faculty asks:

"Where do you validate stock?"

→ `StoreService`

"Where is the product stored?"

→ `InMemoryRepository<Product>`

"How do you make the repository reusable?"

→ `IRepository<T>` + generics

"How do you generate reports without loops?"

→ LINQ

"How do you simulate a database?"

→ `Task.Delay(500)` inside async repository methods

"How do you prevent invalid numeric input from crashing?"

→ `TryParse()`

"How do you handle insufficient stock?"

→ Custom `OutOfStockException`

"Why use decimal?"

→ Monetary values require appropriate precision.

That is the complete process and the architecture you should follow.

YOU

Explain me creation of project, files and classes, namespaces, solution , assembly from scratch for this project I have visual studio downloaded

CHATGPT

Yes. Let's start **completely from zero in Visual Studio**, and before writing the actual application logic, understand exactly what **Solution → Project → Assembly → Namespace → Folder → Class → File** means.

For this assignment, I recommend we build it step-by-step rather than dumping all the code at once.

1. First understand the hierarchy

For your Store Inventory project, think of it like this:

```

Solution
├── Project
│   ├── Assembly
│   ├── Namespaces
│   │   ├── Models
│   │   ├── Interfaces
│   │   ├── Repositories
│   │   ├── Services
│   │   └── Exceptions
│   └── .cs files
│       └── Product.cs

```

```

├── Customer.cs
├── Order.cs
└── ...

```

But there is an important distinction:

Folders and namespaces are not inherently the same thing in C#.

You can have a folder called `Models`, and a namespace called `StoreInventory.Models`, but the compiler doesn't require the folder and namespace to match.

We'll make them match because it keeps the project organized.

2. What is a Solution?

In Visual Studio, a **Solution** is a container for one or more projects.

For this assignment:

```

StoreInventorySolution
├──
└── StoreInventory

```

The solution can eventually contain multiple projects:

```

StoreInventorySolution
├──
├── StoreInventory
├── StoreInventory.Tests
└── StoreInventory.Core

```

But we don't need that complexity right now.

We'll have:

```

Solution
  ↓
1 Project

```

3. What is a Project?

A project is the actual application you're building.

For us:

```

StoreInventory

```

It will contain:

```

Models
Interfaces
Repositories
Services

```

```
Exceptions
Program.cs
```

The project also contains configuration information such as:

```
StoreInventory.csproj
```

This `.csproj` file tells .NET things like:

- Which .NET version you're targeting
- What type of application this is
- Dependencies/packages
- Compilation settings

4. What is an Assembly?

This is an important C# concept.

When you build your project, C# source code:

```
Product.cs
Customer.cs
Order.cs
Program.cs
...
```

gets compiled.

The result is an **assembly**.

For a normal modern .NET application, you'll see something like:

```
StoreInventory.dll
```

and possibly:

```
StoreInventory.exe
```

depending on the application/output/runtime.

Conceptually:

```
Product.cs
Customer.cs
Order.cs
Program.cs
    |
    | C# Compiler
    ↓
StoreInventory.dll
```

So:

Project is what you work on. Assembly is compiled output produced from the project.

5. What is a namespace?

A namespace is a way of logically organizing C# types.

For example:

```
namespace StoreInventory.Models
{
    public class Product
    {
    }
}
```

Here:

```
StoreInventory.Models
```

is the namespace.

And:

```
Product
```

is the class.

The fully qualified name is:

```
StoreInventory.Models.Product
```

Think of it like an address.

6. Folder vs Namespace

This is where Java and C# can initially feel confusing.

In Java you might have:

```
com.company.store
    ↓
folder structure
    ↓
Product.java
```

and the package declaration:

```
package com.company.store;
```

C# is more flexible.

You could physically have:

```
RandomFolder/
    Product.cs
```

while writing:

```
namespace StoreInventory.Models
{
    public class Product
    {
    }
}
```

That is legal.

However, we'll follow the convention:

```
Folder:
Models/

Namespace:
StoreInventory.Models
```

So they visually correspond.

7. What is a class?

A class defines a type/object.

For example:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Here:

```
Product
```

is a class.

An object can then be created:

```
Product laptop = new Product();
```

For this assignment, our main classes will be:

```
Product
Customer
Order
OrderItem
InMemoryRepository
StoreService
ReportService
```

8. What is a `.cs` file?

A `.cs` file contains C# source code.

For example:

```
Product.cs
```

may contain:

```
namespace StoreInventory.Models;

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
}
```

Unlike Java, C# doesn't require:

```
public class Product
```

to be in a file named:

```
Product.cs
```

It's a strong convention, though, and we'll follow it.

9. Now let's actually create the project

Open **Visual Studio**.

You should see something similar to:

```
Create a new project
```

Click:

Create a new project

10. Select Console App

You'll get a template selection screen.

Search:

```
Console App
```

You may see several options.

Choose:

Console App

with C#.

You want the standard:

```
Console App
C#
.NET
```

Do **not** choose:

- Console App Framework
- Console App (.NET Framework)
- Class Library
- ASP.NET Core Web App

For this assignment, choose the normal modern:

Console App

Then click:

Next

11. Configure your project

You'll see something similar to:

```
Project name:
Location:
Solution name:
```

Use something like:

```
Project name:
StoreInventory

Solution name:
StoreInventorySolution
```

You can put the location wherever you normally keep your Visual Studio projects.

For example:

```
D:\Projects\
```

if you prefer keeping development projects on D:.

You previously asked about keeping things on D:, and this is a good place to apply that idea: **the source project can live on D:** without affecting how .NET functions.

12. Important: Solution name vs Project name

Set:

```
Solution name:
StoreInventorySolution
```

and:

```
Project name:
StoreInventory
```

Then you'll get:

```
StoreInventorySolution
|
└─ StoreInventory
```

This distinction is useful.

Solution

```
StoreInventorySolution
```

is the Visual Studio container.

Project

```
StoreInventory
```

is your actual console application.

13. Framework selection

You'll then probably see:

```
Framework:
```

Choose an installed **LTS** version if available.

For example, if Visual Studio offers:

```
.NET 10.0 (LTS)
```

you can use it.

If your course specifically requires another version, follow the course requirement.

For this assignment, there is nothing here that requires an old .NET version.

Keep other options at their defaults unless you know you need something else.

Then click:

Create

14. Visual Studio will create something like this

In **Solution Explorer**, you'll see:

```

Solution 'StoreInventorySolution'
├── StoreInventory
│   ├── Dependencies
│   ├── Program.cs
│   └── StoreInventory.csproj

```

Depending on your Visual Studio/.NET version, the initial project may look slightly different.

15. What is `Program.cs` ?

This is the entry point of your console application.

Initially, modern .NET may give you:

```
Console.WriteLine("Hello, World!");
```

When you run:

```
Ctrl + F5
```

or:

```
F5
```

the application starts executing there.

We'll eventually make `Program.cs` responsible mainly for:

```

Application startup
    ↓
Create services/repositories
    ↓
Start menu

```

We don't want to put **everything** in `Program.cs`.

16. Run the empty project now

Before creating anything else:

Press:

```
Ctrl + F5
```

You should see:

```
Hello, World!
```

This proves:

```

Visual Studio
  ↓
Project
  ↓
C# compiler
  ↓
Build
  ↓
Assembly
  ↓
Run

```

is working.

17. Now let's understand the files Visual Studio created

Right-click the project:

```
StoreInventory
```

and choose:

Open Folder in File Explorer

You'll see something similar to:

```

StoreInventory/
|
├─ Program.cs
├─ StoreInventory.csproj
├─ bin/
└─ obj/

```

Don't worry about `bin` and `obj` yet.

18. What is `.csproj`?

Open:

```
StoreInventory.csproj
```

You'll see XML similar to:

```

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>

```

Your version may differ.

For example:

```
net8.0
net9.0
net10.0
```

depending on what you selected.

This file describes your project.

19. What is `bin` ?

After building, you'll see:

```
bin/
```

This contains compiled output.

Something like:

```
bin/
├── Debug/
│   └── net10.0/
│       ├── StoreInventory.dll
│       ├── StoreInventory.exe
│       └── ...
```

This is where the compiled application goes.

Remember:

```
.cs files
  ↓
compiler
  ↓
.dll / executable
```

20. What is `obj` ?

`obj` contains intermediate build information generated by .NET.

You generally don't manually edit it.

Think:

```
Source code
  ↓
obj
  ↓
bin
```

For now:

Ignore `bin` and `obj`.

Visual Studio/.NET manages them.

21. Now create our folders

In Solution Explorer:

Right-click:

StoreInventory

Select:

Add
↓
New Folder

Create:

Models

Then repeat for:

Interfaces
Repositories
Services
Exceptions

Your project becomes:

```
StoreInventory
|
├── Dependencies
|
├── Models
|
├── Interfaces
|
├── Repositories
|
├── Services
|
├── Exceptions
|
├── Program.cs
└── StoreInventory.csproj
```

This is our starting structure.

22. Create the Product class

Now:

Right-click:

Models

Choose:

Add

↓

Class

Enter:

Product.cs

Click **Add**.

Visual Studio creates:

Models/
Product.cs

23. Add the namespace

You can write:

```
namespace StoreInventory.Models;

public class Product
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public decimal Price { get; set; }

    public int StockQuantity { get; set; }
}
```

Now look at the relationship:

```
Folder
  ↓
Models

Namespace
  ↓
StoreInventory.Models

Class
  ↓
Product
```

```
File
↓
Product.cs
```

Therefore:

```
StoreInventory.Models.Product
```

is the full type name.

24. Why does namespace start with StoreInventory?

Because the project is called:

```
StoreInventory
```

So we'll use:

```
StoreInventory.Models
StoreInventory.Interfaces
StoreInventory.Repositories
StoreInventory.Services
StoreInventory.Exceptions
```

This gives us a clean organization.

25. Create Customer

Right-click:

```
Models
→ Add
→ Class
```

Name:

```
Customer.cs
```

Then:

```
namespace StoreInventory.Models;

public class Customer
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";
}
```

Now:

```
Models
├──
├── Product.cs
├── Customer.cs
```

Namespace:

```
StoreInventory.Models
```

Both classes belong to the same namespace.

26. Create OrderStatus

Right-click:

```
Models
→ Add
→ Class
```

You could create a class and change it to enum, but Visual Studio may also offer an enum template depending on version.

Name:

```
OrderStatus.cs
```

Write:

```
namespace StoreInventory.Models;

public enum OrderStatus
{
    Pending,
    Confirmed,
    Cancelled,
    Completed
}
```

Now:

```
OrderStatus
```

is not a class.

It is an:

```
enum
```

27. Create OrderItem

Create:

Models/OrderItem.cs

with:

```
namespace StoreInventory.Models;

public class OrderItem
{
    public int ProductId { get; set; }

    public int Quantity { get; set; }

    public decimal UnitPrice { get; set; }
}
```

28. Create Order

Create:

Models/Order.cs

with:

```
namespace StoreInventory.Models;

public class Order
{
    public int Id { get; set; }

    public int CustomerId { get; set; }

    public List<OrderItem> Items { get; set; } = new();

    public DateTime OrderDate { get; set; }

    public decimal TotalAmount { get; set; }

    public decimal Discount { get; set; }

    public OrderStatus Status { get; set; }
}
```

Now our Models folder is:

```
Models
|
├── Product.cs
├── Customer.cs
├── Order.cs
├── OrderItem.cs
└── OrderStatus.cs
```

29. One important thing: `List<OrderItem>`

Look at:

```
public List<OrderItem> Items { get; set; } = new();
```

This means:

An Order contains multiple OrderItems.

So:

```
Order
├── CustomerId
└── Items
    ├── Laptop × 1
    ├── Mouse × 2
    └── Keyboard × 1
```

This is why we don't put just one `ProductId` directly into the Order.

30. Now create Interfaces folder

Right-click:

```
Interfaces
→ Add
→ Class
```

Name it:

```
IRepository.cs
```

Then we'll turn it into an interface.

```
namespace StoreInventory.Interfaces;

public interface IRepository<T>
{
    Task AddAsync(T item);

    Task<T?> GetByIdAsync(int id);

    Task<List<T>> GetAllAsync();

    Task UpdateAsync(T item);

    Task DeleteAsync(int id);
}
```

Notice:

```
IRepository<T>
```

The `T` means it's generic.

31. Why does the name start with I?

C# convention:

```
IService
ILogger
IProductService
```

Interfaces usually start with:

```
I
```

So:

```
IRepository<T>
```

means:

This is an interface representing repository behavior.

32. Create Repository folder

Right-click:

```
Repositories
→ Add
→ Class
```

Name:

```
InMemoryRepository.cs
```

Eventually it will look like:

```
namespace StoreInventory.Repositories;

public class InMemoryRepository<T> : IRepository<T>
{
    ...
}
```

Notice:

```
InMemoryRepository<T> : IRepository<T>
```

means:

InMemoryRepository implements IRepository.

33. Namespace import

Here's where namespaces become useful.

`IRepository<T>` is in:

```
StoreInventory.Interfaces
```

But `InMemoryRepository<T>` is in:

```
StoreInventory.Repositories
```

Therefore, the repository needs to access the interface.

You can write:

```
using StoreInventory.Interfaces;
```

at the top.

Then:

```
namespace StoreInventory.Repositories;

public class InMemoryRepository<T> : IRepository<T>
{
}
```

34. What does `using` mean?

Suppose you have:

```
StoreInventory.Models.Product
```

If you don't import the namespace, you could write:

```
StoreInventory.Models.Product laptop;
```

everywhere.

But with:

```
using StoreInventory.Models;
```

you can simply write:

```
Product laptop;
```

So:

```
using
↓
allows convenient access to types in another namespace
```

35. Create Exceptions folder

Right-click:

```
Exceptions
→ Add
→ Class
```

Name:

```
OutOfStockException.cs
```

We'll eventually have:

```
namespace StoreInventory.Exceptions;

public class OutOfStockException : Exception
{
    public OutOfStockException(string message)
        : base(message)
    {
    }
}
```

So:

```
Exception
↑
OutOfStockException
```

Our custom exception inherits from the built-in .NET `Exception`.

36. Create Services folder

Create:

```
Services
|
├── StoreService.cs
└── ReportService.cs
```

We'll use:

```
StoreService
```

For business operations:

```
Add product
Update product
```

```

Delete product
Add customer
Checkout
Stock validation
Discount

```

ReportService

For:

```

Top-selling products
Revenue/customer
Low stock
Date-range search

```

37. Final Solution Explorer

At this point, your Solution Explorer should look approximately like:

```

Solution 'StoreInventorySolution'
|
├── StoreInventory
│   ├── Dependencies
│   ├── Models
│   │   ├── Product.cs
│   │   ├── Customer.cs
│   │   ├── Order.cs
│   │   ├── OrderItem.cs
│   │   └── OrderStatus.cs
│   ├── Interfaces
│   │   └── IRepository.cs
│   ├── Repositories
│   │   └── InMemoryRepository.cs
│   ├── Services
│   │   ├── StoreService.cs
│   │   └── ReportService.cs
│   ├── Exceptions
│   │   └── OutOfStockException.cs
│   ├── Program.cs
│   └── StoreInventory.csproj

```

That's the structure I recommend for your assignment.

38. Now connect everything mentally

Here's the important part.

Product

```
Models/Product.cs
↓
StoreInventory.Models.Product
```

Customer

```
Models/Customer.cs
↓
StoreInventory.Models.Customer
```

Repository interface

```
Interfaces/IRepository.cs
↓
StoreInventory.Interfaces.IRepository<T>
```

Repository implementation

```
Repositories/InMemoryRepository.cs
↓
StoreInventory.Repositories.InMemoryRepository<T>
```

Service

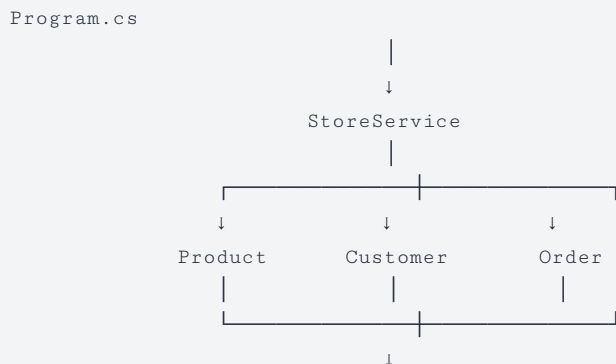
```
Services/StoreService.cs
↓
StoreInventory.Services.StoreService
```

Exception

```
Exceptions/OutOfStockException.cs
↓
StoreInventory.Exceptions.OutOfStockException
```

39. And how everything connects

Eventually:



```

    IRepository<T>
      |
      ↓
    InMemoryRepository<T>
      |
      ↓
    List<T>

```

Reports:

```

Orders
  ↓
ReportService
  ↓
LINQ
  ↓
Reports

```

Exceptions:

```

StoreService
  ↓
Insufficient stock
  ↓
OutOfStockException
  ↓
Program catches it
  ↓
Friendly message

```

40. The most important distinction to remember

Since you asked specifically about **solution**, **assembly**, **namespace**, **class** and **folder**, remember this:

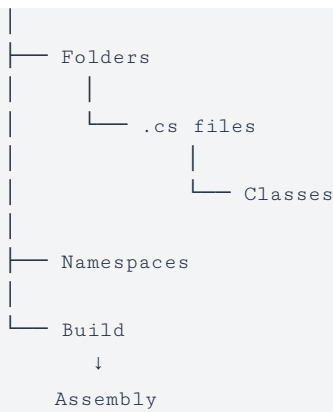
Thing	What it is	Example
Solution	Container for projects	StoreInventorySolution
Project	Actual application/library	StoreInventory
.csproj	Project configuration	StoreInventory.csproj
Folder	Physical organization	Models/
.cs file	Source-code file	Product.cs
Namespace	Logical organization of types	StoreInventory.Models
Class	Type defined in C#	Product
Assembly	Compiled output	StoreInventory.dll

And the relationship:

```

Solution
  |
  └─ Project

```



One subtle but important point

Folder $\neq$ namespace.

We are intentionally making them look related:

```
Models/
  Product.cs

namespace StoreInventory.Models
```

because it is good project organization.

But C# doesn't technically require them to match.

41. Don't write the full application yet

At this point, I recommend **stopping before implementing repository/service logic**.

First make sure your Visual Studio project looks like:

```
StoreInventorySolution
├── StoreInventory
│   ├── Models
│   │   ├── Product.cs
│   │   ├── Customer.cs
│   │   ├── Order.cs
│   │   ├── OrderItem.cs
│   │   └── OrderStatus.cs
│   ├── Interfaces
│   │   └── IRepository.cs
│   ├── Repositories
│   │   └── InMemoryRepository.cs
│   ├── Services
│   │   ├── StoreService.cs
│   │   └── ReportService.cs
│   ├── Exceptions
│   │   └── OutOfStockException.cs
│   └── Program.cs
```

Then we can build the actual application one layer at a time: first the models, then the generic repository, then CRUD, then checkout/business logic, then LINQ reports, and finally the console

menu. This will also make it much easier for you to understand every line rather than just copy-pasting a finished project.

YOU

Tell me the name of folder I should first create on my desktop

CHATGPT

Create **one folder on your Desktop** named:

```
StoreInventoryProject
```

So your Desktop will look like:

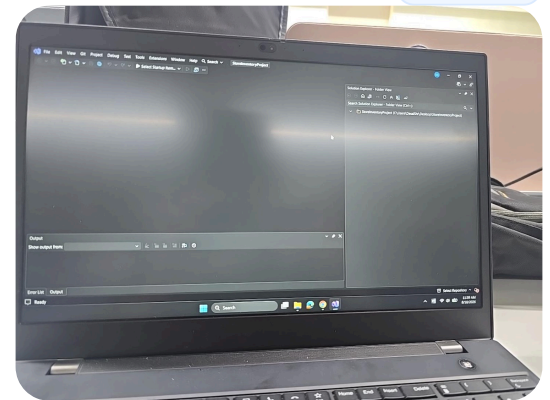
```
Desktop
|
└── StoreInventoryProject
```

Then, **don't manually create any other folders yet.**

We'll open Visual Studio and create the **Solution and Project inside this folder**. After that, I'll tell you exactly what to click and what to name each thing, one step at a time.

YOU

Here I m



CHATGPT

Yes 👍 I can see exactly where you are.

You have opened the folder:

```
C:\Users\...\Desktop\StoreInventoryProject
```

in Visual Studio. **That's correct.** However, right now you have only opened the folder – we haven't created the C# project yet.

Let's do it step by step.

Step 1 – Create the C# project

At the top-left of Visual Studio, click:

File → New → Project

You should get the **Create a new project** window.

Step 2 – Search for Console App

In the search box, type:

Console App

Select:

Console App

Make sure it says **C#** and **.NET**.

Then click **Next**.

Step 3 – Project details

You should now see fields such as:

Project name:
Location:
Solution name:

Enter:

Project name

StoreInventory

Solution name

StoreInventorySolution

For **Location**, select your existing Desktop folder:

C:\Users\<<your-name>\Desktop\StoreInventoryProject

So conceptually we want:

```
Desktop
├── StoreInventoryProject
│   ├── StoreInventorySolution
│   │   └── StoreInventory
```

⚠ One thing

If Visual Studio gives you a checkbox like:

"Place solution and project in the same directory"

leave it unchecked for now.

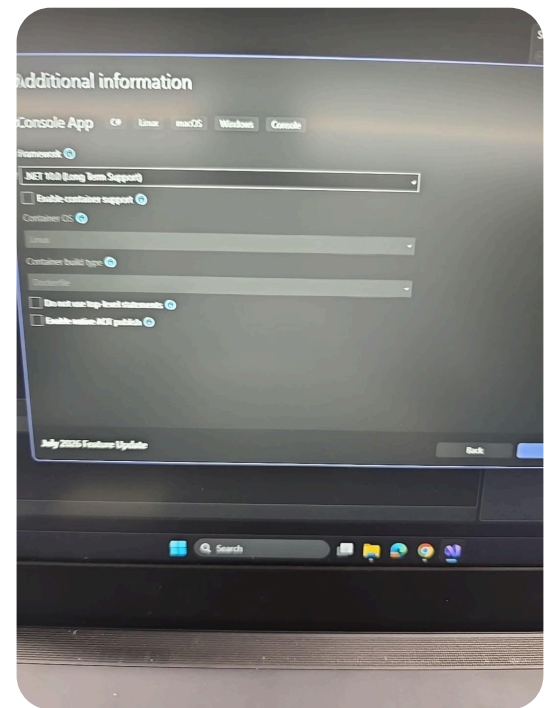
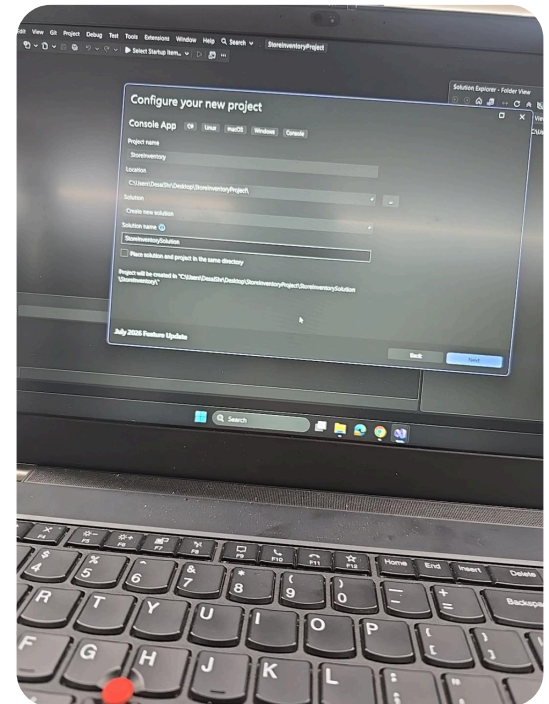
This will help us clearly see the difference between the **Solution** and **Project**, which is exactly what you're trying to learn.

Then click **Next** and **stop there**.

Don't create anything else yet.

Send me a picture of the next screen, and I'll tell you exactly what to select.

YOU



CHATGPT

Perfect. 👍 I can see both screens clearly. **Everything is set correctly.**

You are now on the **Additional Information** screen.

What you have selected

Your screen shows:

- **Project type:** Console App 
- **Framework:** .NET 10.0 (Long Term Support) 
- **Enable container support:** unchecked 
- **Container OS:** disabled — correct
- **Container build type:** disabled — correct
- **Do not use top-level statements:** unchecked
- **Enable native AOT publish:** unchecked

Do NOT change anything

For this project, leave all of these exactly as they are.

Especially:

Framework:

.NET 10.0 (Long Term Support)

That's perfectly fine.

Now click:

Create → bottom-right.

Visual Studio will create the project.

What should happen next?

After you click **Create**, Visual Studio should show something similar to:

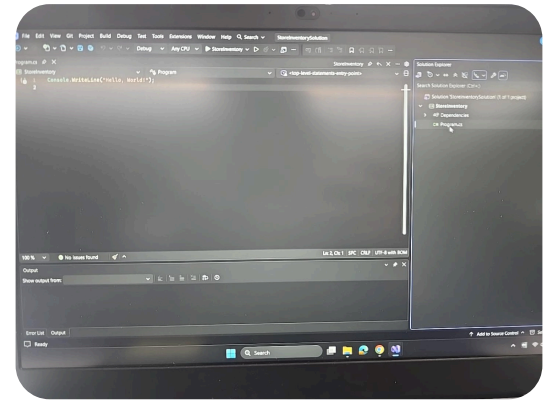
```
Solution 'StoreInventorySolution'
|
├── StoreInventory
│   ├── Dependencies
│   ├── Program.cs
│   └── StoreInventory.csproj
```

You may initially see a slightly different layout because your Solution Explorer is currently in **Folder View**.

Don't create any folders or files yet after clicking Create.

Once the project opens, send me a screenshot of the **Solution Explorer**, and I'll guide you through the next step: **understanding exactly what Visual Studio created and creating our `Models` folder and first `Product.cs` class.**

YOU



CHATGPT

Perfect. ✅ Your project has been created successfully.

I can see:

```
Solution 'StoreInventorySolution'
├── StoreInventory
│   ├── Dependencies
│   └── Program.cs
```

And your `Program.cs` currently contains:

```
Console.WriteLine("Hello, World!");
```

This is exactly what we expect.

Step 1 – Understand what you have right now

You currently have:

Solution

```
StoreInventorySolution
```

This is the Visual Studio container.

Project

```
StoreInventory
```

This is your actual C# console application.

Program.cs

```
Program.cs
```

This is the starting point of your application.

Dependencies

This contains references/dependencies that your project uses. We won't modify it manually right now.

Step 2 – First create the `Models` folder

Now we'll start building our application.

In the **Solution Explorer on the right**:

1. Right-click on **StoreInventory**.
2. Select **Add**.
3. Select **New Folder**.
4. Name the folder:

`Models`

You should then have:

```
StoreInventory
|
├── Dependencies
├── Models
└── Program.cs
```

Stop here.

Don't create `Product.cs` yet.

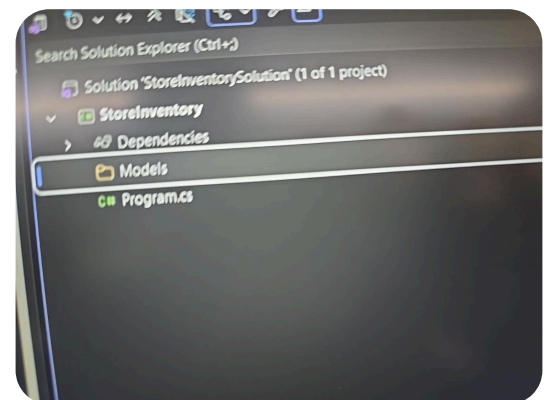
Create only the **Models** folder and send me a screenshot.

Then I'll show you exactly how to create the first class:

```
Models
└── Product.cs
```

and explain what **class + file + namespace** mean while we create it.

YOU



CHATGPT

Perfect ✅ you created the `Models` folder correctly.

Your current structure is:

```

Solution 'StoreInventorySolution'
├── StoreInventory
│   ├── Dependencies
│   ├── Models ← our folder
│   └── Program.cs

```

Now let's create our **first class**.

Step 2 — Create `Product.cs`

In the Solution Explorer:

1. **Right-click** `Models`
2. Select **Add**
3. Select **Class...**
4. In the name box, enter:

```
Product.cs
```

1. Click **Add**

Visual Studio should create:

```

Models
├── Product.cs

```

Step 3 — What Visual Studio will give you

You'll probably see something like:

```

namespace StoreInventory.Models;

internal class Product
{
}

```

or possibly:

```

namespace StoreInventory.Models
{
    internal class Product
    {
    }
}

```

Don't worry about the exact formatting.

Change it to:

```

namespace StoreInventory.Models;

public class Product

```

```
{
}
```

Why `public`?

`Product` will be used by other parts of our application, such as:

```
StoreService
Repository
Program
Reports
```

So we want the class accessible outside its immediate context.

Step 4 – Add the Product properties

Now put this inside the class:

```
namespace StoreInventory.Models;

public class Product
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public decimal Price { get; set; }

    public int StockQuantity { get; set; }
}
```

So your complete `Product.cs` should be exactly that.

What did we just create?

This is important to understand rather than just copy.

1. `Product.cs`

This is the **C# source file**.

```
Product.cs
```

2. `Product`

This is the **class** inside that file.

```
public class Product
```

3. `StoreInventory.Models`

This is the **namespace**.

```
namespace StoreInventory.Models;
```

So the full name of our class is:

```
StoreInventory.Models.Product
```

4. Models

This is the **folder** in Visual Studio.

```
Models/
```

We are intentionally using the same logical naming:

```
Folder:    Models
Namespace: StoreInventory.Models
Class:     Product
File:      Product.cs
```

What do the properties mean?

Our product needs four pieces of information:

```
public int Id { get; set; }
```

Product's unique ID.

Example:

```
101
```

```
public string Name { get; set; } = "";
```

Product name.

Example:

```
Laptop
```

```
public decimal Price { get; set; }
```

Product price.

Example:

```
60000.00
```

We use `decimal` because this is money.

```
public int StockQuantity { get; set; }
```

How many units are available.

Example:

10

So one actual product object could represent:

```
Product
-----
Id:           101
Name:         Laptop
Price:        ₹60,000
StockQuantity: 10
```

Your Solution Explorer should now look like:

```
Solution 'StoreInventorySolution'
|
├── StoreInventory
│   ├── Dependencies
│   ├── Models
│   │   └── Product.cs
│   └── Program.cs
```

Do this now and send me a screenshot of `Product.cs` open in the editor. Then we'll create `Customer.cs`, and I'll explain why `Customer` is another class in the same namespace.

YOU

next

CHATGPT

Great. Now we move to the **next model**: `Customer`.

Step 3 – Create `Customer.cs`

In **Solution Explorer**:

1. Right-click **Models**
2. Select **Add → Class...**
3. Name it:

`Customer.cs`

1. Click **Add**.

You should now have:

```
StoreInventory
|
├── Dependencies
```



Step 4 – Write the Customer class

Open `Customer.cs` and replace its contents with:

```

namespace StoreInventory.Models;

public class Customer
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public string Email { get; set; } = "";

    public string Phone { get; set; } = "";
}
  
```

What does this represent?

A customer in our store has:

```

Customer
-----
Id
Name
Email
Phone
  
```

For example:

```

Id:      1
Name:    Shreyash
Email:    shreyash@gmail.com
Phone:    9876543210
  
```

Why is Customer another class?

Because **Product** and **Customer** are two different types of objects in our application.

We could technically put everything into one class, but that would be bad design.

Instead:

```

StoreInventory.Models
|
|— Product
  
```



Customer

Both belong to:

```
namespace StoreInventory.Models;
```

but they represent different things.

This is one of the basic ideas of **Object-Oriented Programming**:

Each class represents a meaningful entity in the application.

Step 5 – Create `OrderStatus.cs`

Now right-click:

Models

→ **Add**

→ **Class...**

Name it:

`OrderStatus.cs`

Then replace everything with:

```
namespace StoreInventory.Models;

public enum OrderStatus
{
    Pending,
    Confirmed,
    Cancelled,
    Completed
}
```

Notice something different

Previously we had:

```
public class Product
```

and:

```
public class Customer
```

Now we have:

```
public enum OrderStatus
```

An **enum** represents a fixed set of possible values.

An order can be:

```
Pending
Confirmed
Cancelled
Completed
```

Instead of allowing someone to type arbitrary text such as:

```
"banana"
"xyz"
"hello"
```

we restrict the status to these predefined values.

Step 6 — Create `OrderItem.cs`

Right-click:

```
Models
→ Add
→ Class...
```

Name:

```
OrderItem.cs
```

Put:

```
namespace StoreInventory.Models;

public class OrderItem
{
    public int ProductId { get; set; }

    public int Quantity { get; set; }

    public decimal UnitPrice { get; set; }
}
```

Why do we need OrderItem?

Imagine a customer buys:

```
Laptop × 1
Mouse × 2
Keyboard × 1
```

Each line is an **OrderItem**.

So:

```

Order
├── OrderItem
│   ProductId = Laptop
│   Quantity = 1
├── OrderItem
│   ProductId = Mouse
│   Quantity = 2
└── OrderItem
    ProductId = Keyboard
    Quantity = 1

```

That's why an order will eventually contain:

```
List<OrderItem>
```

Step 7 – Create `Order.cs`

Again:

Models → Add → Class...

Name:

```
Order.cs
```

Use:

```

namespace StoreInventory.Models;

public class Order
{
    public int Id { get; set; }

    public int CustomerId { get; set; }

    public List<OrderItem> Items { get; set; } = new();

    public DateTime OrderDate { get; set; }

    public decimal TotalAmount { get; set; }

    public decimal Discount { get; set; }

    public OrderStatus Status { get; set; }
}

```

Now understand the relationship

We have created:

```

Models
├── Product.cs
│   └── Product
├── Customer.cs
│   └── Customer
├── OrderItem.cs
│   └── OrderItem
├── Order.cs
│   └── Order
└── OrderStatus.cs
    └── OrderStatus

```

And **all five** belong to the same namespace:

```
StoreInventory.Models
```

Most important relationship so far

An `Order` contains `OrderItem` s:

```
public List<OrderItem> Items { get; set; } = new();
```

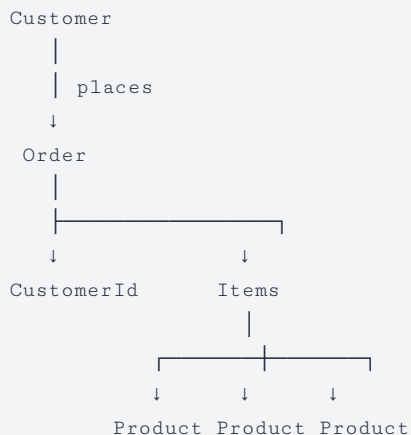
And each `OrderItem` refers to a product:

```
public int ProductId { get; set; }
```

An `Order` also belongs to a customer:

```
public int CustomerId { get; set; }
```

So conceptually:



This is the basic data model of our store.

Your Solution Explorer should now be

```

StoreInventorySolution
|
├── StoreInventory
│   ├── Dependencies
│   ├── Models
│   │   ├── Product.cs
│   │   ├── Customer.cs
│   │   ├── OrderItem.cs
│   │   ├── Order.cs
│   │   └── OrderStatus.cs
│   └── Program.cs

```

Do these five files now.

Once they're created, **don't create the next folders yet**. The next step will be creating the `Interfaces` folder and, importantly, understanding **what an interface actually is and why `IRepository<T>` is generic** before we implement it.

YOU

Done , next

CHATGPT

Great.  Now we move from **Models** to the **Interface layer**.

Step 8 — Create the `Interfaces` folder

In **Solution Explorer**:

1. Right-click **StoreInventory** — not `Models`.
2. Select **Add → New Folder**.
3. Name it:

```
Interfaces
```

Your structure should become:

```

StoreInventory
|
├── Dependencies
├── Interfaces
├── Models
│   ├── Product.cs
│   ├── Customer.cs
│   ├── OrderItem.cs
│   ├── Order.cs
│   └── OrderStatus.cs

```

Program.cs

Step 9 – What is an Interface?

Before creating the file, understand this because it is one of the important requirements in your assignment.

An interface is basically a **contract**.

Suppose we say:

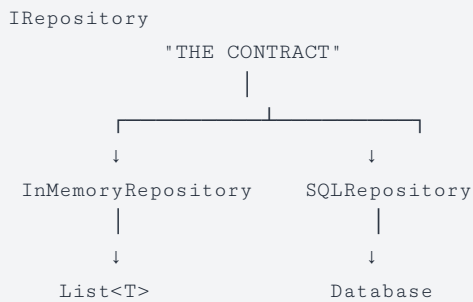
```
 IRepository
```

The contract says:

Any repository must know how to Add, Get, Update and Delete data.

But the interface doesn't say **how** those operations are performed.

Think:



Both can follow the same contract.

For our assignment, we'll implement:

```
 IRepository<T>
    ↓
 InMemoryRepository<T>
```

Step 10 – Create IRepository.cs

Right-click:

```
 Interfaces
```

Then:

Add → Class...

Name it:

```
 IRepository.cs
```

Click **Add**.

Step 11 – Change it to an interface

Visual Studio may generate something like:

```
namespace StoreInventory.Interfaces;

internal class IRepository
{
}
```

Delete everything inside the file and put:

```
namespace StoreInventory.Interfaces;

public interface IRepository<T>
{
    Task AddAsync(T item);

    Task<T?> GetByIdAsync(int id);

    Task<List<T>> GetAllAsync();

    Task UpdateAsync(T item);

    Task DeleteAsync(int id);
}
```

Save with:

Ctrl + S

Step 12 – Understand every part

Let's break this down.

Namespace

```
namespace StoreInventory.Interfaces;
```

This tells C#:

`IRepository` belongs to the `StoreInventory.Interfaces` namespace.

Interface

```
public interface IRepository<T>
```

`IRepository` is the name.

The `I` at the beginning is a C# naming convention meaning:

This is an interface.

What is `<T>` ?

This is **generics**.

We don't want to create separate interfaces like:

```
IProductRepository
ICustomerRepository
IOrderRepository
```

Instead, we create one reusable interface:

```
IRepository<T>
```

Then:

```
IRepository<Product>
```

means:

Repository for Products.

And:

```
IRepository<Customer>
```

means:

Repository for Customers.

And:

```
IRepository<Order>
```

means:

Repository for Orders.

Same interface. Different type.

Step 13 – Understand the first method

```
Task AddAsync(T item);
```

This says:

Any repository must provide an asynchronous method for adding an item.

If:

```
T = Product
```

then it becomes conceptually:

```
Task AddAsync(Product item);
```

If:

```
T = Customer
```

then it becomes:

```
Task AddAsync(Customer item);
```

That's the power of generics.

Step 14 – Get one item

```
Task<T?> GetByIdAsync(int id);
```

This means:

Get an object by its ID.

For a Product repository:

```
GetByIdAsync(101)
    ↓
Product
```

For a Customer repository:

```
GetByIdAsync(1)
    ↓
Customer
```

The `?` means the result **may be null** if no matching object exists.

For example:

```
Search Product ID 999
    ↓
No product found
    ↓
null
```

Step 15 – Get all

```
Task<List<T>> GetAllAsync();
```

This means:

Return a list containing all objects.

For products:

```
List<Product>
```

For customers:

```
List<Customer>
```

Again, `T` determines the type.

Step 16 – Update

```
Task UpdateAsync(T item);
```

This says:

A repository must be able to update an existing item.

Step 17 – Delete

```
Task DeleteAsync(int id);
```

This says:

A repository must be able to delete an item using its ID.

Step 18 – Why are these `Task`?

The assignment specifically requires:

Use `async/await` and `Task.Delay(500)` inside data methods to simulate cloud-database delays.

Our repository will eventually pretend that it is communicating with a database.

For example:

```
StoreService
  ↓
Repository
  ↓
Wait 500 ms
  ↓
Pretend database responded
  ↓
Return data
```

That's why the repository methods are asynchronous.

We'll implement the actual `Task.Delay(500)` when we create `InMemoryRepository`.

Step 19 – What we've achieved

Your project now has:

```
StoreInventory
├── Interfaces
│   └── IRepository.cs
├── Models
│   ├── Product.cs
│   ├── Customer.cs
│   ├── OrderItem.cs
│   ├── Order.cs
│   └── OrderStatus.cs
└── Program.cs
```

And conceptually:

```
IRepository<T>
    |
    | "CRUD contract"
    |
    ↓
InMemoryRepository<T>
    |
    ↓
List<T>
```

We haven't created `InMemoryRepository` yet.

That's the next step.

Before moving on

Save all files:

Ctrl + Shift + S isn't necessary; just **Ctrl + S** is enough.

Then look at the bottom of Visual Studio.

You should ideally see:

```
No issues found
```

If there are no errors, we're ready.

Next: `Repositories/InMemoryRepository.cs`

There we'll take this abstract interface:

```
IRepository<T>
```

and actually implement it using:

```
List<T>
```

That's where you'll see **generics + interface implementation + async/await + Task.Delay(500)** come together.

YOU

Okay next

CHATGPT

Great. Now we create the **Repository layer**. This is where your `IRepository<T>` interface becomes an actual working component.

Step 10 – Create the `Repositories` folder

In **Solution Explorer**:

1. Right-click **StoreInventory**
2. Select **Add → New Folder**
3. Name it:

```
Repositories
```

Your structure should now be:

```
StoreInventory
|
|— Dependencies
|— Interfaces
|   |— IRepository.cs
|
|— Models
|   |— Product.cs
|   |— Customer.cs
|   |— OrderItem.cs
|   |— Order.cs
|   |— OrderStatus.cs
|
|— Repositories
|
|— Program.cs
```

Step 11 – Create `InMemoryRepository.cs`

Right-click:

```
Repositories
```

→ **Add**

→ **Class...**

Name it:

InMemoryRepository.cs

Click **Add**.

Step 12 — Replace the generated code

Delete whatever Visual Studio generated and put:

```
using StoreInventory.Interfaces;

namespace StoreInventory.Repositories;

public class InMemoryRepository<T> : IRepository<T>
{
    private readonly List<T> items = new();

    public async Task AddAsync(T item)
    {
        await Task.Delay(500);

        items.Add(item);
    }

    public async Task<T?> GetByIdAsync(int id)
    {
        await Task.Delay(500);

        return default;
    }

    public async Task<List<T>> GetAllAsync()
    {
        await Task.Delay(500);

        return items.ToList();
    }

    public async Task UpdateAsync(T item)
    {
        await Task.Delay(500);
    }

    public async Task DeleteAsync(int id)
    {
        await Task.Delay(500);
    }
}
```

But don't worry: this is intentionally not the final implementation. We're going to improve `GetByIdAsync`, `UpdateAsync`, and `DeleteAsync` in the next step.

First understand what we just created.

Step 13 – The most important line

Look at:

```
public class InMemoryRepository<T> : IRepository<T>
```

This means:

```
InMemoryRepository<T>
    |
    | implements
    ↓
IRepository<T>
```

The `:` means **implements/inherits from**, depending on what is on the right side.

Because `IRepository<T>` is an interface, this means:

`InMemoryRepository<T>` promises to implement everything required by `IRepository<T>`.

That's why Visual Studio expects these five methods:

```
AddAsync()
GetByIdAsync()
GetAllAsync()
UpdateAsync()
DeleteAsync()
```

Step 14 – What does "InMemory" mean?

Normally, an application could store products in:

```
SQL Server
PostgreSQL
MongoDB
MySQL
```

But your assignment specifically says:

Use in-memory storage.

So we're going to use:

```
private readonly List<T> items = new();
```

This is effectively our temporary database.

For example, if:

```
T = Product
```

then:

```
List<T>
```

becomes:

```
List<Product>
```

And if:

```
T = Customer
```

it becomes:

```
List<Customer>
```

Step 15 – Why `private readonly` ?

```
private readonly List<T> items = new();
```

`private`

Only the repository itself can directly access the list.

Other parts of the application shouldn't do:

```
repository.items.Add(...)
```

They should go through the repository methods.

`readonly`

The `items` variable cannot be replaced with another list after construction.

We can still do:

```
items.Add(item);
```

but we can't do:

```
items = new List<T>();
```

Step 16 – `AddAsync`

We have:

```
public async Task AddAsync(T item)
{
    await Task.Delay(500);

    items.Add(item);
}
```

The process is:

```
AddAsync(product)
    ↓
Wait 500 ms
    ↓
items.Add(product)
```

Why wait?

Because we're pretending:

```
Application
    ↓
Cloud database
    ↓
Network
    ↓
Database
```

takes some time.

Step 17 — Why `async`?

Because we're using:

```
await Task.Delay(500);
```

A method containing `await` generally needs to be asynchronous.

Hence:

```
public async Task AddAsync(...)
```

The important pattern is:

```
async
+
await
+
Task
```

Step 18 — `GetAllAsync`

```
public async Task<List<T>> GetAllAsync()
{
    await Task.Delay(500);

    return items.ToList();
}
```

This means:

1. Wait 500 ms.
2. Take all objects.
3. Return them.

Why:

```
items.ToList()
```

instead of:

```
return items;
```

Because we don't want callers to directly manipulate our internal list.

So we're returning a copy.

Step 19 — Why this is generic

This one class:

```
InMemoryRepository<T>
```

can work with all our models.

For products:

```
InMemoryRepository<Product>
```

For customers:

```
InMemoryRepository<Customer>
```

For orders:

```
InMemoryRepository<Order>
```

So instead of writing:

```
ProductRepository
CustomerRepository
OrderRepository
```

we have one reusable class.

That's exactly why the assignment asks for a **Generic Repository**.

Step 20 — One problem we need to solve

Look at:

```
public async Task<T?> GetByIdAsync(int id)
```

Our generic `T` doesn't necessarily have an `Id` property.

For example, C# doesn't know that:

`T`

has:

`Id`

So we can't simply write:

```
items.FirstOrDefault(x => x.Id == id)
```

because the compiler doesn't know that `T` has `Id`.

We need to solve this properly.

There are several approaches.

For this assignment, I recommend using an **interface for identifiable entities**.

That will also reinforce the "interfaces for structure" requirement.

Step 21 – Create `IEntity.cs`

Go to:

`Interfaces`

Right-click:

Add → Class...

Name:

`IEntity.cs`

Replace the contents with:

```
namespace StoreInventory.Interfaces;

public interface IEntity
{
    int Id { get; set; }
}
```

Now we're saying:

Anything that is stored in our generic repository must have an `Id`.

Step 22 – Modify Product

Open:

`Models/Product.cs`

Change:

```
public class Product
```

to:

```
public class Product : IEntity
```

You'll need:

```
using StoreInventory.Interfaces;
```

at the top.

So:

```
using StoreInventory.Interfaces;

namespace StoreInventory.Models;

public class Product : IEntity
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public decimal Price { get; set; }

    public int StockQuantity { get; set; }
}
```

Step 23 — Modify Customer

Open:

```
Customer.cs
```

Change:

```
public class Customer
```

to:

```
public class Customer : IEntity
```

So:

```
using StoreInventory.Interfaces;

namespace StoreInventory.Models;

public class Customer : IEntity
{
```

```

public int Id { get; set; }

public string Name { get; set; } = "";

public string Email { get; set; } = "";

public string Phone { get; set; } = "";
}

```

Step 24 – Modify Order

Open:

```
Order.cs
```

Change:

```
public class Order
```

to:

```
public class Order : IEntity
```

Add:

```
using StoreInventory.Interfaces;
```

So the beginning becomes:

```

using StoreInventory.Interfaces;

namespace StoreInventory.Models;

public class Order : IEntity
{

```

We don't need to modify `OrderItem` because we won't use it as a repository entity.

Step 25 – Now modify IRepository

Open:

```
Interfaces/IRepository.cs
```

Change:

```
public interface IRepository<T>
```

to:

```
public interface IRepository<T> where T : IEntity
```

So the complete interface becomes:

```
namespace StoreInventory.Interfaces;

public interface IRepository<T> where T : IEntity
{
    Task AddAsync(T item);

    Task<T?> GetByIdAsync(int id);

    Task<List<T>> GetAllAsync();

    Task UpdateAsync(T item);

    Task DeleteAsync(int id);
}
```

What does this mean?

This:

```
where T : IEntity
```

means:

T must implement `IEntity`.

Therefore:

<code>IRepository<Product></code>	✓
<code>IRepository<Customer></code>	✓
<code>IRepository<Order></code>	✓
<code>IRepository<string></code>	✗

because `string` doesn't implement our `IEntity`.

This is called a **generic constraint**.

Step 26 — Now modify InMemoryRepository

Change:

```
public class InMemoryRepository<T> : IRepository<T>
```

to:

```
public class InMemoryRepository<T> : IRepository<T>
    where T : IEntity
```

So:

```
using StoreInventory.Interfaces;
```

```

namespace StoreInventory.Repositories;

public class InMemoryRepository<T> : IRepository<T>
    where T : IEntity
{
    private readonly List<T> items = new();

    public async Task AddAsync(T item)
    {
        await Task.Delay(500);

        items.Add(item);
    }

    public async Task<T?> GetByIdAsync(int id)
    {
        await Task.Delay(500);

        return items.FirstOrDefault(x => x.Id == id);
    }

    public async Task<List<T>> GetAllAsync()
    {
        await Task.Delay(500);

        return items.ToList();
    }

    public async Task UpdateAsync(T item)
    {
        await Task.Delay(500);

        var existingItem = items.FirstOrDefault(x => x.Id == item.Id);

        if (existingItem != null)
        {
            items.Remove(existingItem);
            items.Add(item);
        }
    }

    public async Task DeleteAsync(int id)
    {
        await Task.Delay(500);

        var item = items.FirstOrDefault(x => x.Id == id);

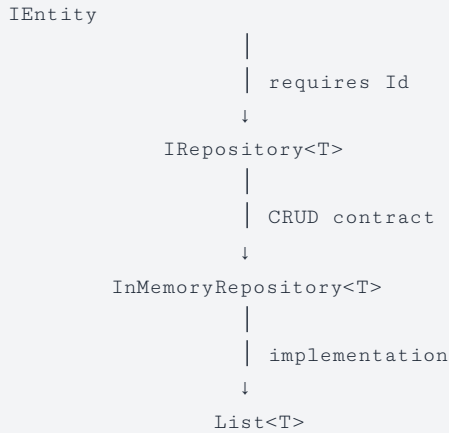
        if (item != null)
        {
            items.Remove(item);
        }
    }
}

```

Now our generic repository actually works.

Step 27 – Understand the complete repository

We now have:



For products:

```

InMemoryRepository<Product>
    ↓
List<Product>
  
```

For customers:

```

InMemoryRepository<Customer>
    ↓
List<Customer>
  
```

For orders:

```

InMemoryRepository<Order>
    ↓
List<Order>
  
```

Step 28 – Build the project

Now press:

Ctrl + Shift + B

This means:

Build Solution.

Look at the bottom of Visual Studio.

You want:

```
Build succeeded.
```

or:

0 Errors

If you get errors, **don't try random fixes**. Send me a screenshot of the Error List and we'll fix them.

Your structure now

You should have:

```

StoreInventorySolution
├── StoreInventory
│   ├── Dependencies
│   ├── Interfaces
│   │   ├── IEntity.cs
│   │   └── IRepository.cs
│   ├── Models
│   │   ├── Product.cs
│   │   ├── Customer.cs
│   │   ├── OrderItem.cs
│   │   ├── Order.cs
│   │   └── OrderStatus.cs
│   ├── Repositories
│   │   └── InMemoryRepository.cs
│   └── Program.cs

```

Stop after the build. The next step will be creating the `Exceptions` folder and `OutOfStockException`, then we'll create the **StoreService**, where the actual checkout/business rules from your assignment will start coming together.

YOU

Okay next

 CHATGPT

Great. Now we move to **exception handling**, which is one of the explicit requirements of your assignment.

Step 13 – Create the `Exceptions` folder

In **Solution Explorer**:

1. Right-click **StoreInventory**
2. Select **Add → New Folder**
3. Name it:

Exceptions

Your structure should now be:

```

StoreInventory
├── Dependencies
├── Interfaces
│   ├── IEntity.cs
│   └── IRepository.cs
├── Models
│   ├── Product.cs
│   ├── Customer.cs
│   ├── OrderItem.cs
│   ├── Order.cs
│   └── OrderStatus.cs
├── Repositories
│   └── InMemoryRepository.cs
├── Exceptions
└── Program.cs
  
```

Step 14 – Create `OutOfStockException.cs`

Right-click:

Exceptions

→ **Add** → **Class...**

Name:

`OutOfStockException.cs`

Replace the generated code with:

```

namespace StoreInventory.Exceptions;

public class OutOfStockException : Exception
{
    public OutOfStockException(string message)
        : base(message)
    {
    }
}
  
```

Save with **Ctrl + S**.

Step 15 – Understand what we created

We have:

```

Exception
  ↑
  |
OutOfStockException

```

Our class:

```
public class OutOfStockException : Exception
```

inherits from the built-in .NET:

```
Exception
```

This means `OutOfStockException` is a specialized exception specifically for our store.

Why do we need a custom exception?

Suppose our inventory contains:

```
Laptop
Stock = 2
```

Customer tries to buy:

```
Quantity = 5
```

That's a business-specific error.

We don't want to simply say:

```
Something went wrong.
```

We want:

```
Insufficient stock for Laptop.
Available: 2
Requested: 5
```

So our service will eventually do:

```
throw new OutOfStockException(
    $"Insufficient stock for {product.Name}. " +
    $"Available: {product.StockQuantity}, " +
    $"Requested: {quantity}."
);
```

Then `Program.cs` will catch it.

Step 16 – Understand `throw`

This:

```
throw new OutOfStockException("...");
```

means:

Stop the current operation and report this exception to whoever called the method.

The flow will eventually be:

```
Customer requests 5 laptops
    ↓
  StoreService
    ↓
  Check stock
    ↓
  Stock = 2
    ↓
  5 > 2
    ↓
  throw OutOfStockException
    ↓
  Program
    ↓
  catch (...)
    ↓
  Display error message
    ↓
  Show menu again
```

The application doesn't need to terminate.

Step 17 – Now create the `Services` folder

Right-click:

```
StoreInventory
```

→ **Add** → **New Folder**

Name it:

```
Services
```

Your project now becomes:

```
StoreInventory
|
├── Dependencies
|
├── Interfaces
|   ├── IEntity.cs
|   └── IRepository.cs
|
├── Models
|   └── Product.cs
```

```

|   |   | Customer.cs
|   |   | OrderItem.cs
|   |   | Order.cs
|   |   | OrderStatus.cs
|
|   | Repositories
|   |   | InMemoryRepository.cs
|
|   | Exceptions
|   |   | OutOfStockException.cs
|
|   | Services
|
|   | Program.cs

```

Step 18 – Create `StoreService.cs`

Right-click:

Services

→ Add → Class...

Name:

StoreService.cs

For now, put this:

```

using StoreInventory.Exceptions;
using StoreInventory.Interfaces;
using StoreInventory.Models;

namespace StoreInventory.Services;

public class StoreService
{
    private readonly IRepository<Product> productRepository;
    private readonly IRepository<Customer> customerRepository;
    private readonly IRepository<Order> orderRepository;

    public StoreService(
        IRepository<Product> productRepository,
        IRepository<Customer> customerRepository,
        IRepository<Order> orderRepository)
    {
        this.productRepository = productRepository;
        this.customerRepository = customerRepository;
        this.orderRepository = orderRepository;
    }
}

```

Step 19 – This is an important C# concept: Dependency Injection

Look at:

```
public StoreService(
    IRepository<Product> productRepository,
    IRepository<Customer> customerRepository,
    IRepository<Order> orderRepository)
```

We're saying:

StoreService needs three repositories to work.

It needs:

```
Product Repository
Customer Repository
Order Repository
```

Instead of creating them inside `StoreService`, we're **giving them to it**.

This is called **dependency injection**.

Step 20 – Why not do this?

We could have written:

```
private readonly InMemoryRepository<Product> productRepository
    = new();
```

But that would tightly couple `StoreService` to:

```
InMemoryRepository
```

Instead we use:

```
IRepository<Product>
```

So the service only knows about the **interface**.

Conceptually:

```
StoreService
      |
      ↓
    IRepository<Product>
      ↑
      |
    InMemoryRepository
```

Later, if you wanted:

SQLRepository

you could theoretically replace the implementation without rewriting the service.

Step 21 — Add Product CRUD methods

Now inside `StoreService`, below the constructor, add:

```
public async Task AddProductAsync(Product product)
{
    await productRepository.AddAsync(product);
}

public async Task<List<Product>> GetProductsAsync()
{
    return await productRepository.GetAllAsync();
}

public async Task<Product?> GetProductAsync(int id)
{
    return await productRepository.GetByIdAsync(id);
}

public async Task UpdateProductAsync(Product product)
{
    await productRepository.UpdateAsync(product);
}

public async Task DeleteProductAsync(int id)
{
    await productRepository.DeleteAsync(id);
}
```

These methods are basically forwarding the operations to the repository.

The architecture is now:

```
Program
  ↓
StoreService
  ↓
IRepository<Product>
  ↓
InMemoryRepository<Product>
  ↓
List<Product>
```

Step 22 — Add Customer CRUD methods

Below the product methods, add:

```
public async Task AddCustomerAsync(Customer customer)
{
    await customerRepository.AddAsync(customer);
}
```

```

}

public async Task<List<Customer>> GetCustomersAsync()
{
    return await customerRepository.GetAllAsync();
}

public async Task<Customer?> GetCustomerAsync(int id)
{
    return await customerRepository.GetByIdAsync(id);
}

public async Task UpdateCustomerAsync(Customer customer)
{
    await customerRepository.UpdateAsync(customer);
}

public async Task DeleteCustomerAsync(int id)
{
    await customerRepository.DeleteAsync(id);
}

```

Now `StoreService` can handle:

```

Products → CRUD
Customers → CRUD

```

Step 23 — Now comes the important part: Checkout

The assignment says:

Validate stock before checking out.

We'll create a method:

```
CheckoutAsync()
```

But before writing it, understand what it needs.

When the customer checks out:

```

Customer ID
    ↓
Order Items
    ↓
Find customer
    ↓
Find every product
    ↓
Validate stock
    ↓
Calculate total
    ↓
Apply discount
    ↓

```

```

Reduce stock
↓
Create Order
↓
Save Order

```

Step 24 — Add Checkout method

Add this to `StoreService`:

```

public async Task<Order> CheckoutAsync(
    int customerId,
    List<OrderItem> items)
{
    var customer = await customerRepository.GetByIdAsync(customerId);

    if (customer == null)
    {
        throw new Exception("Customer not found.");
    }

    var products = new List<Product>();

    foreach (var item in items)
    {
        var product = await productRepository.GetByIdAsync(item.ProductId);

        if (product == null)
        {
            throw new Exception(
                $"Product with ID {item.ProductId} not found.");
        }

        if (item.Quantity <= 0)
        {
            throw new Exception(
                "Quantity must be greater than zero.");
        }

        if (product.StockQuantity < item.Quantity)
        {
            throw new OutOfStockException(
                $"Insufficient stock for {product.Name}. " +
                $"Available: {product.StockQuantity}, " +
                $"Requested: {item.Quantity}.");
        }

        products.Add(product);
    }

    decimal subtotal = 0;

    for (int i = 0; i < items.Count; i++)
    {
        items[i].UnitPrice = products[i].Price;
    }
}

```

```

        subtotal += products[i].Price * items[i].Quantity;
    }

    decimal discount = subtotal > 5000
        ? subtotal * 0.10m
        : 0;

    decimal total = subtotal - discount;

    for (int i = 0; i < items.Count; i++)
    {
        products[i].StockQuantity -= items[i].Quantity;

        await productRepository.UpdateAsync(products[i]);
    }

    var order = new Order
    {
        CustomerId = customerId,
        Items = items,
        OrderDate = DateTime.Now,
        TotalAmount = total,
        Discount = discount,
        Status = OrderStatus.Completed
    };

    await orderRepository.AddAsync(order);

    return order;
}

```

Important

Your assignment says **the four reports should not use manual loops**.

The loops above are for **checkout processing**, not the reports. That's fine.

The reports will be written using LINQ later.

Step 25 – Understand the checkout validation

This part:

```

if (product.StockQuantity < item.Quantity)
{
    throw new OutOfStockException(...);
}

```

is the key requirement.

For example:

```

Stock = 10
Requested = 3

```

```
10 < 3 → false
```

Continue.

But:

```
Stock = 2
Requested = 5

2 < 5 → true
```

Throw:

```
OutOfStockException
```

Step 26 — Why do we validate BEFORE reducing stock?

This is critical.

Suppose an order contains:

```
Laptop × 1
Mouse × 2
Keyboard × 5
```

and keyboard has only 2.

We first check:

```
Laptop    ✓
Mouse     ✓
Keyboard  ✗
```

Only if **everything passes** do we execute:

```
products[i].StockQuantity -= items[i].Quantity;
```

So we don't partially modify the inventory.

That's good business logic.

Step 27 — Discount logic

Our code:

```
decimal discount = subtotal > 5000
    ? subtotal * 0.10m
    : 0;
```

is the ternary operator.

It's equivalent to:

```
decimal discount;

if (subtotal > 5000)
{
    discount = subtotal * 0.10m;
}
else
{
    discount = 0;
}
```

So:

```
₹4,000
  ↓
No discount

₹5,000
  ↓
No discount

₹5,001
  ↓
10% discount
```

because the requirement says **over ₹5,000**.

Step 28 — Build the project

Now press:

Ctrl + Shift + B

You should get:

```
Build succeeded
```

If there are errors, don't change anything randomly.

Send me the screenshot of the **Error List**.

Current project structure

You should now have:

```
StoreInventorySolution
├── StoreInventory
│   ├── Dependencies
│   ├── Interfaces
│   ├── IEntity.cs
│   └── IRepository.cs
```

```
|
|— Models
|   |— Product.cs
|   |— Customer.cs
|   |— OrderItem.cs
|   |— Order.cs
|   |— OrderStatus.cs
|
|— Repositories
|   |— InMemoryRepository.cs
|
|— Exceptions
|   |— OutOfStockException.cs
|
|— Services
|   |— StoreService.cs
|
|— Program.cs
```

What's left after this?

We're getting close to the actual application:

1. ReportService
↓
2. Program.cs setup
↓
3. Console menu
↓
4. TryParse input handling
↓
5. Product CRUD UI
↓
6. Customer CRUD UI
↓
7. Order/Checkout UI
↓
8. Four LINQ reports
↓
9. Exception handling in UI
↓
10. Full testing

For now, build the project and confirm that it compiles.